Transformers: 2017 vs 2026
(Multimodal LLMs, MoE, Attention at Scale)

Naeemullah Khan
naeemullah.khan@kaust.edu.sa

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

LMH
Lady Margaret Hall
University of Oxford

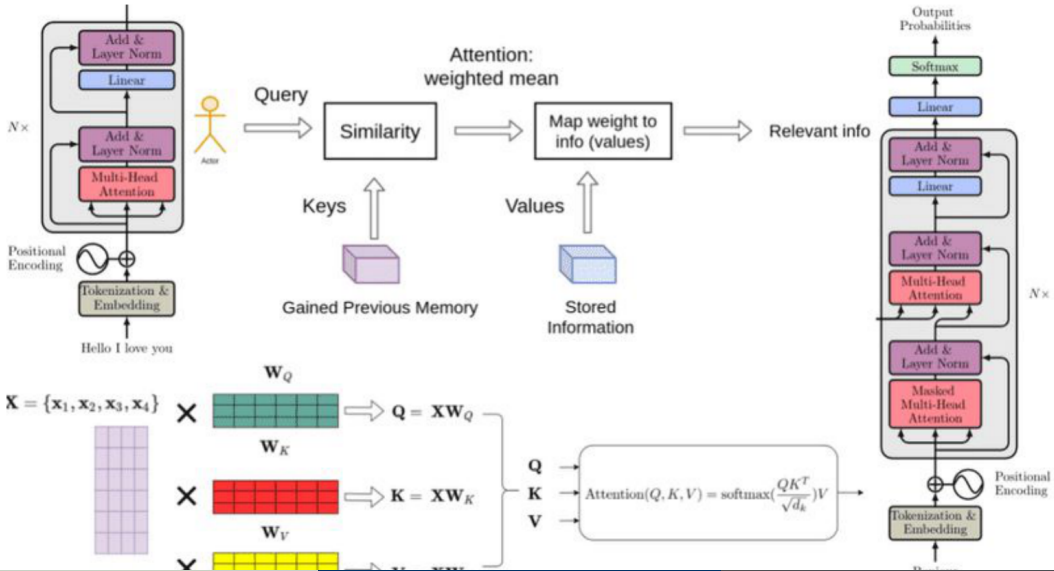
June 12, 2026

Table of Contents

Four Key Engineering Changes (2017 → 2026)

1. **Bandwidth matters more than just speed:** New attention methods like FlashAttention focus on moving data efficiently and using memory well (built-in memory or RAG based retrieval).
2. **From fixed to flexible compute:** Models now adjust their computation and memory use based on the input (using tools like RoPE, sparse MoE, and adaptive resolution).
3. **From separate to unified input streams:** Instead of having different pieces for text, images, etc., everything is processed together in one sequence—with new practical challenges.
4. **More thinking during use:** Some new models use extra internal steps ("thinking tokens") for harder questions, shifting cost to longer decoding instead of just bigger models.

The 2017 Transformer



Why the Transformer?

- The original 2017 Transformer was designed to address key limitations of Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) units:
  - Lack of parallelization
  - Vanishing gradient problem inherent in sequential processing
- Replaced recurrence with self-attention to allow all tokens in a sequence to be processed simultaneously during training[1]
- Enabled significant reductions in wall-clock time needed for model convergence

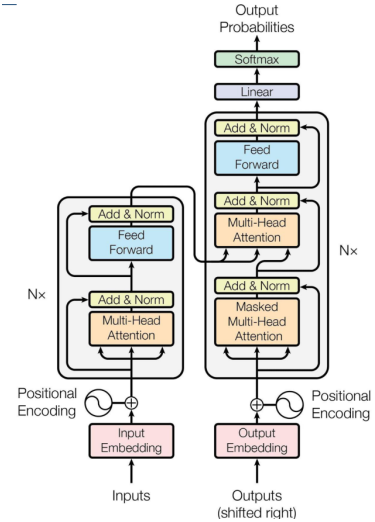
[1] See Vaswani et al., "Attention is All You Need," 2017.

Introduction to Transformers

- **What is a Transformer?**
  - Introduced by Vaswani et al. (2017) – "Attention is All You Need"
- **Key features:**
  - No recurrence, all attention
  - Encoder-decoder structure
  - Enables parallel processing
  - Scales well with data and compute

Transformer Architecture: Components Overview

- Tokenization
- Input Embeddings
- Position Encodings
- Query, Key, & Value
- Attention
- Self Attention
- Multi-Head Attention
- Feed Forward
- Add & Norm
- Encoders
- Masked Attention
- Encoder Decoder Attention
- Linear
- Softmax
- Decoders
- Encoder-Decoder Models



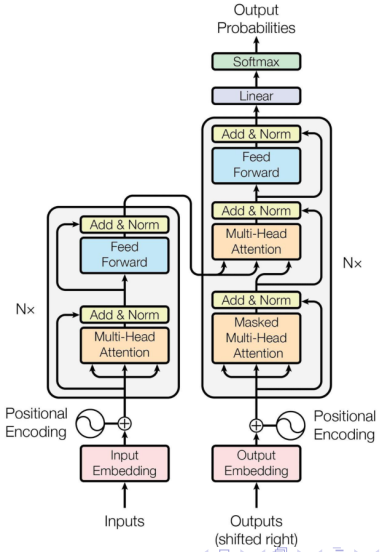
Machine Translation

Targets

Ich habe einen
Apfel gegessen

Inputs

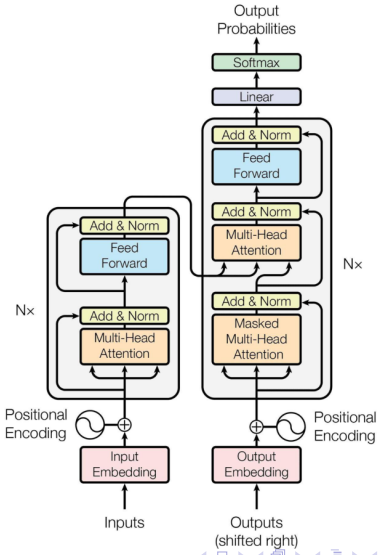
I ate an apple



Processing Inputs

Inputs

I ate an apple



Self-Attention Mechanism

- Computes contextual representation of a word using all other words
- **Inputs:** Query (Q), Key (K), Value (V)
- **Output:** Weighted sum of values

Attention formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

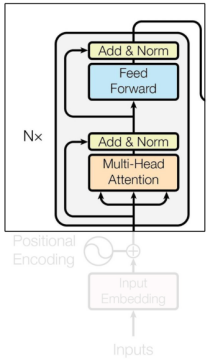
- Enables each word to attend to all others

Self-Attention Intuition: The "Bank" Analogy

- **Query:** What we're looking for (e.g., the meaning of "bank" in context)
- **Keys:** Tags or features associated with each word/sentence (e.g., "river", "money")
- **Values:** Information linked to each key (e.g., context-specific meaning)
- Uses dot-product similarity between Query and Keys to assign attention weights
- Weighted sum of Values produces the contextual representation

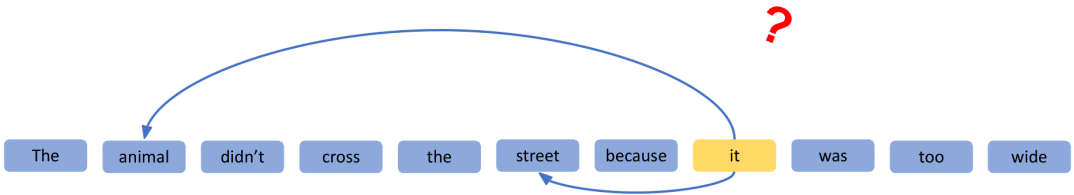
Self Attention

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$



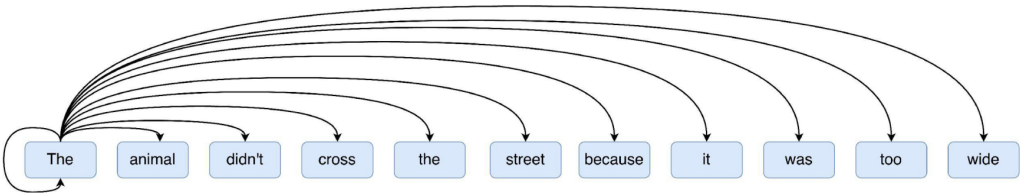
Self Attention

Self Attention

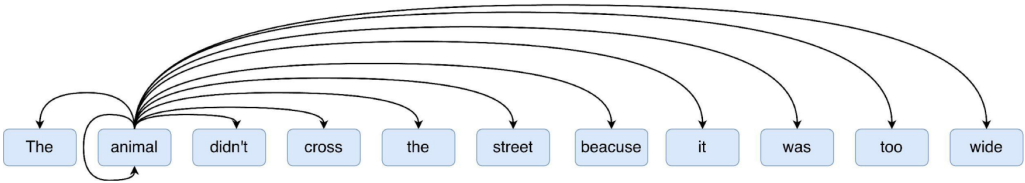


coreference resolution?

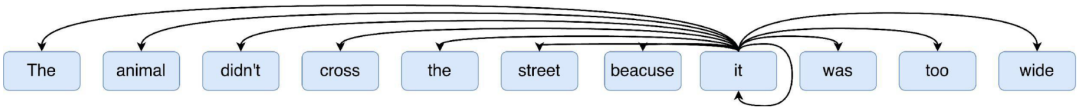
Self Attention



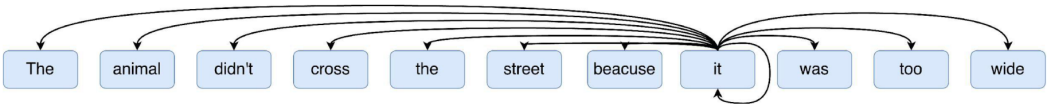
Self Attention



Self Attention



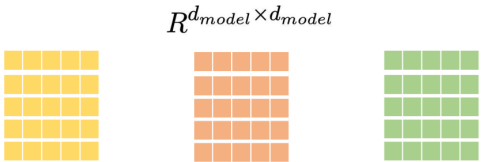
Self Attention

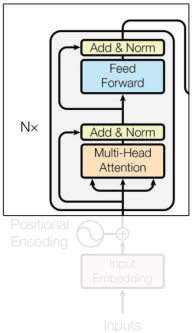


SELF

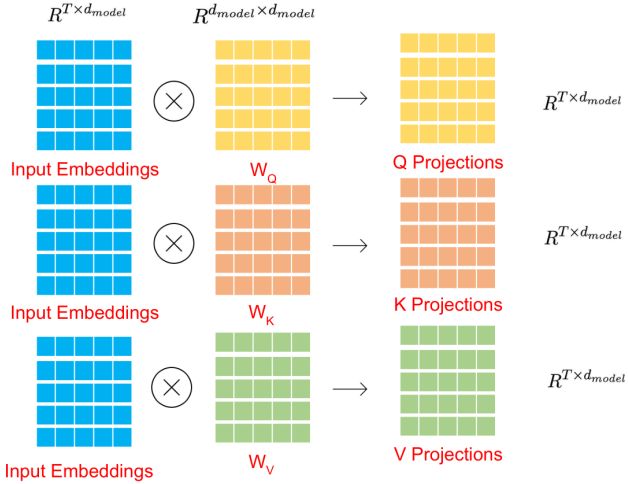
Query Inputs = Key Inputs = Value Inputs

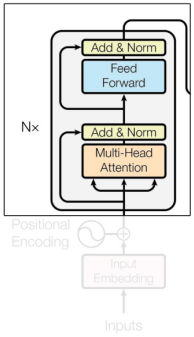
Self Attention



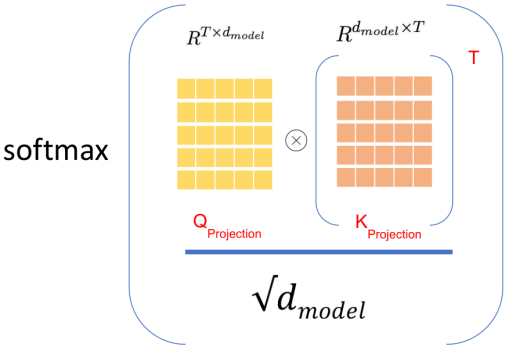


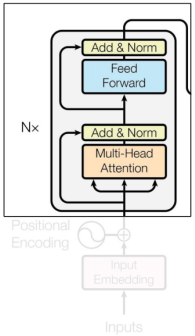
Self Attention



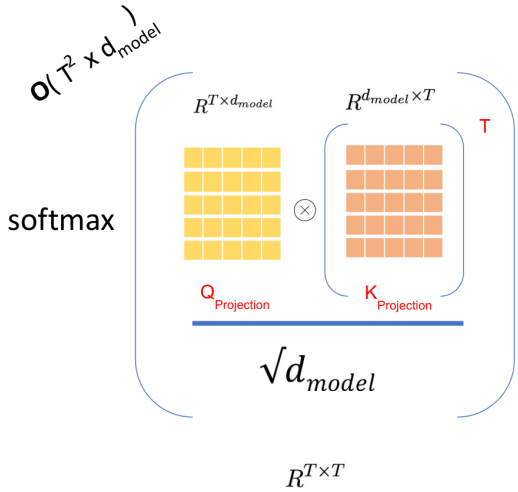


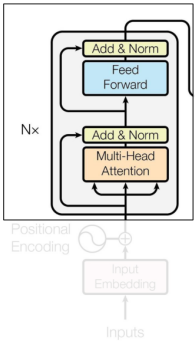
Self Attention



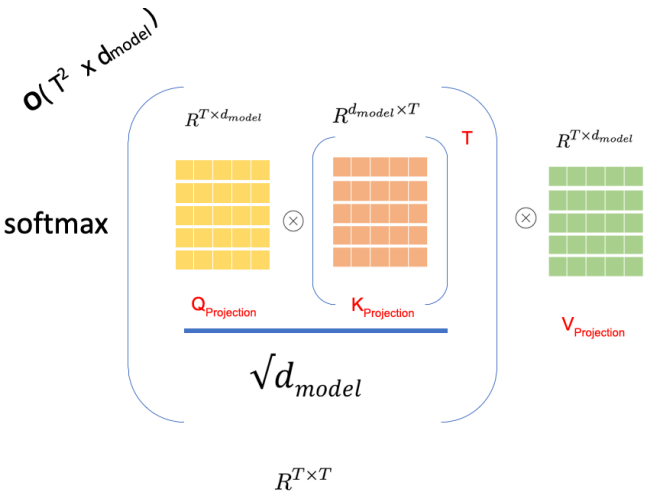


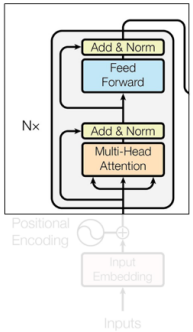
Self Attention





Self Attention

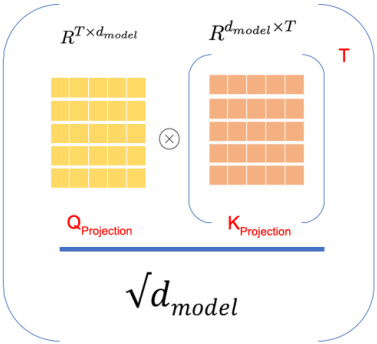




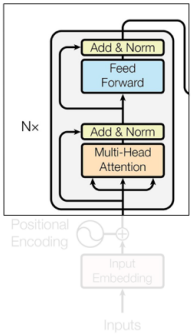
Self Attention

$o(T^2 \times d_{model})$

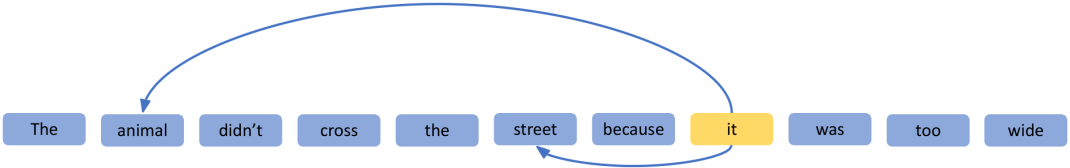
softmax



$R^{T \times T}$

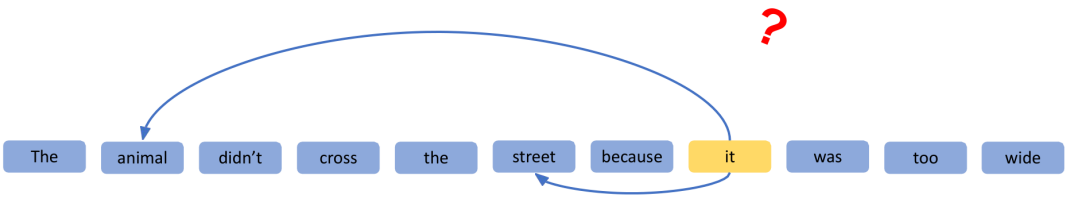


Self Attention



Coreference resolution ✓

Self Attention



Sentence boundaries ?

Coreference resolution ✓

Context ?

Semantic relationships ?

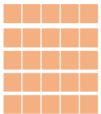
Part of Speech ?

Comparisons ?

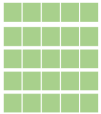
Self Attention

$$R^{d_{model} \times d_{model}}$$

W_Q

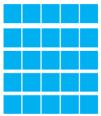


W_K

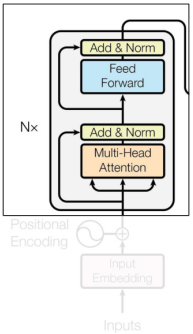


W_v

$$R^{T \times d_{model}}$$



Input Embeddings



Multi-Head Attention

- Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions.
- It consists of multiple attention heads, each with its own set of parameters.
- The outputs of all heads are concatenated and linearly transformed.

Formula:

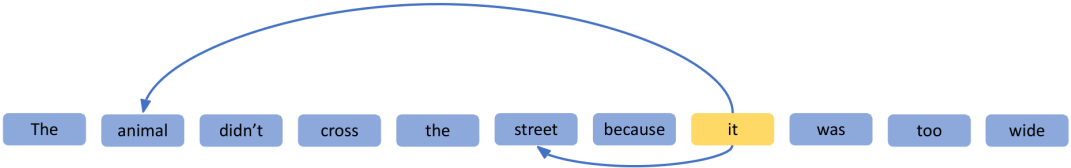
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O$$

where each head is defined as:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

- W_i^Q, W_i^K, and W_i^V are learned projection matrices for each head.
- W^O is the output projection matrix that combines the outputs of all heads.

Multi-Head Attention

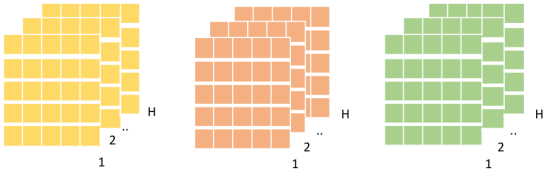


Sentence boundaries ✓ Coreference resolution ✓ Context ✓

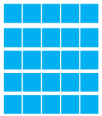
Semantic relationships ✓ Part of speech ✓ Comparisons ✓

Multi-Head Attention

$R^{d_{model} \times d_h}$



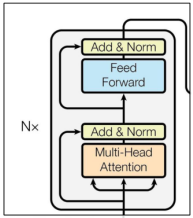
$W_{Q1,}$ $W_{Q2,\ \ldots}$ $W_{QH,}$ $W_{K1,}$ $W_{K2,\ \ldots}$ $W_{KH,}$ $W_{V1,}$ $W_{V2,\ \ldots}$ $W_{VH,}$



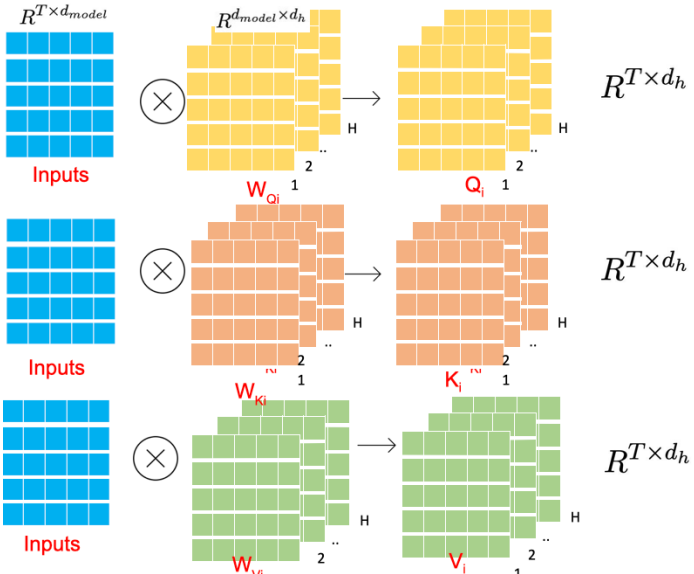
Input Embeddings

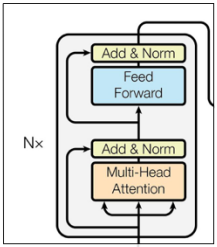
$R^{T \times d_{model}}$

$$d_h = \frac{d_{model}}{h}$$

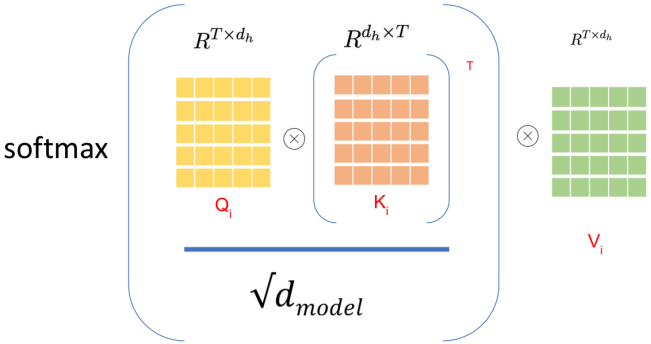


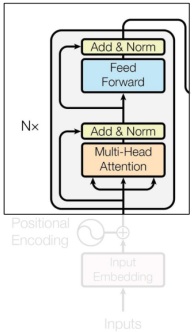
Multi-Head Attention





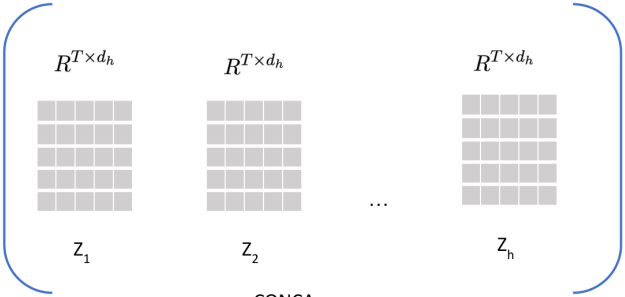
Multi-Head Attention

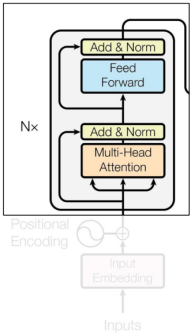




for all i ∈ [1, h]

Multi-Head Attention



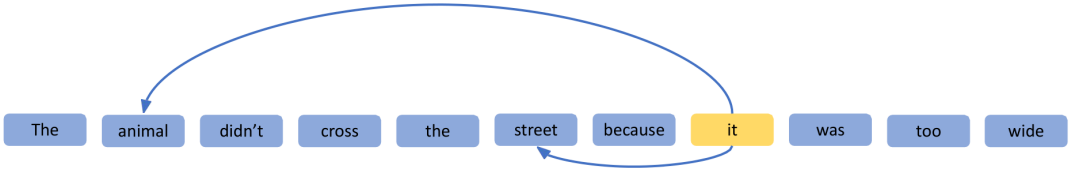


Multi Head Attention : Z

$$d_h = \frac{d_{model}}{h}$$

$$R^{T \times d_{model}}$$

Multi-Head Attention



Sentence boundaries ✓ Coreference resolution ✓ Context ✓

Semantic relationships ✓ Part of speech ✓ Comparisons ✓

Benefits of Multi-Head Attention

- Enables the model to jointly attend to information from different representation subspaces at different positions.
- Captures various linguistic and syntactic patterns by using multiple attention heads.
- Improves the model's ability to represent complex relationships in the data.
- Provides richer and more diverse feature representations.

Positional Encoding

- Transformers lack sequence order awareness.
- **Positional encodings** are added to input embeddings to provide information about token positions.
- **Sinusoidal encoding:**

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

Position Encoding

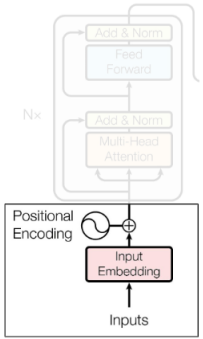
For each position, an embedded input is moved the same distance but at a different angle. **Inputs that are close to each other in the sequence have similar perturbations, but inputs that are far apart are perturbed in different directions.**

$$PE_{(pos,\,2i)} = \sin\left(pos/10000^{2i/d_{\text{model}}}\right)$$

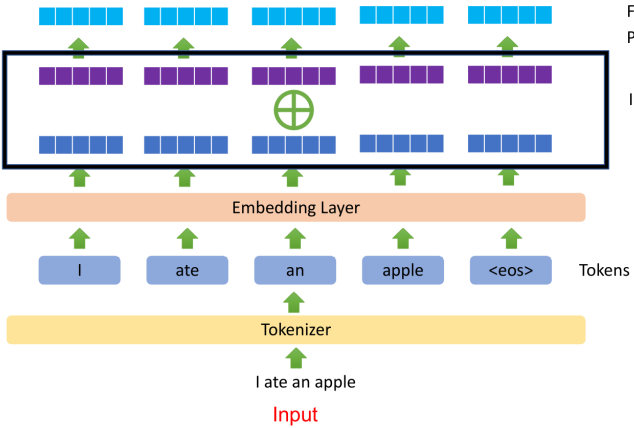
$$PE_{(pos,\,2i+1)} = \cos\left(pos/10000^{2i/d_{\text{model}}}\right)$$

pos idx of the token in input sentence
i i^{th} dimension out of d
d_{model} embedding dimension of each token

Different calculations for odd and even embedding indices.



Position Encoding



Alternatives to Positional Encoding

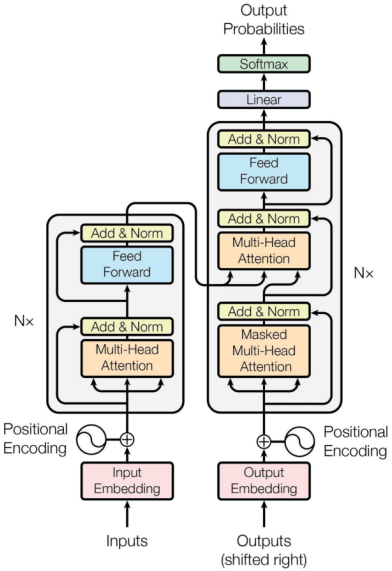
- **Learnable position embeddings:**
  - Position embeddings are parameters learned during training.
  - Allow the model to adapt position representations to the task.
- **Rotary Position Embeddings (RoPE):**
  - Encodes positions by rotating query and key vectors in multi-head attention.
  - Enables better extrapolation to longer sequences.
- **ALiBi (Attention with Linear Biases):**
  - Adds linear biases to attention scores based on distance between tokens.
  - Encourages attention to nearby tokens without explicit position embeddings.

Transformer Block

- **Multi-head Attention**
- **Add & Norm**
- **Feed-forward Layer**
- **Add & Norm**

Note
Residual connections and layer normalization are crucial for effective training.

Transformer Block



Pre-LayerNorm vs. Post-LayerNorm

Feature	Pre-LN	Post-LN
Stability	✓ More stable	✗ Less stable
Gradient flow	✓ Better	✗ Can explode/vanish
Used in	GPT-3, T5	Original Transformer

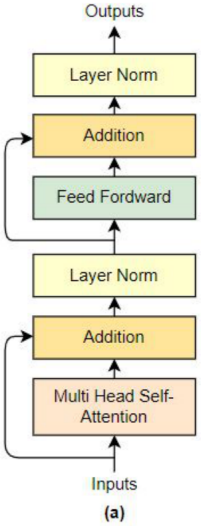
Table 1: Comparison of Pre-LayerNorm and Post-LayerNorm Architectures

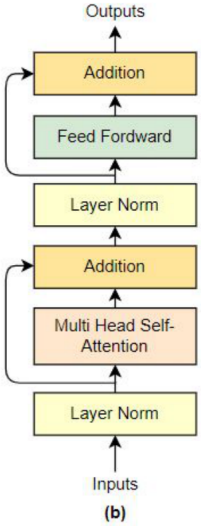
Pre-LN: LayerNorm applied **before** sublayers

Post-LN: LayerNorm applied **after** sublayers

Pre-LayerNorm vs. Post-LayerNorm

Post-LN

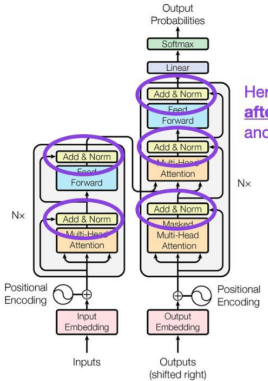




Pre-LN

Pre-LayerNorm vs. Post-LayerNorm

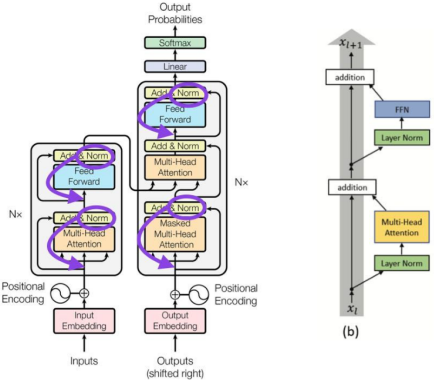
Post-LN Transformer



Here, LayerNorms are **after** attention and **after** fully connected layers

Better gradients if LayerNorm is placed inside residual connection, before the attention and fully connected layers

Pre-LN Transformer



Places the layer normalization between the residual blocks: the expected gradients of the parameters near the output layer are large

Layer Normalization vs RMSNorm

LayerNorm: Normalizes across the feature dimension by subtracting the mean and dividing by the standard deviation.

RMSNorm: Removes the mean and scales by the root mean square (RMS) of the features.

- Simpler
- Faster
- Slightly better in large-scale settings.

Key-Value Caching

- In **decode**, the model emits **one token per step**, but that token still depends on the **key and value** tensors for **all** earlier positions.
- Those positions include the **prompt** (K/V from **prefill**) plus any **new** K/V computed up to the current step.
- **KV caching** keeps these tensors in GPU memory so they are **not recomputed** at every step; each iteration **appends** the newly computed K/V to the cache for the next step.
- In many stacks there is **one KV cache per transformer layer**.

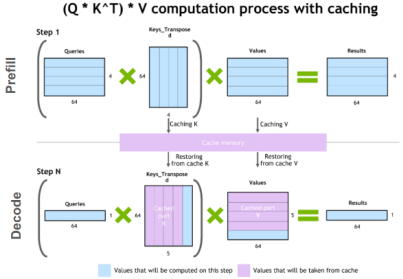


Figure 1: illustration of the key-value caching mechanism.

Verma & Vaidya, "Mastering LLM Techniques: Inference Optimization," NVIDIA Technical Blog (2023). Article — Key-value caching (Figure 1).

KV Cache Memory

- Major GPU memory uses for serving: **model weights** and the **KV cache** (self-attention tensors cached to skip redundant work).
- Under batching, each request still needs a **separate** KV-cache allocation—often the limiting factor before weights alone.
- Cost scales **linearly** with batch and length—caps throughput and long-context use; motivates MQA/GQA, paging (PagedAttention), KV quantisation, etc.

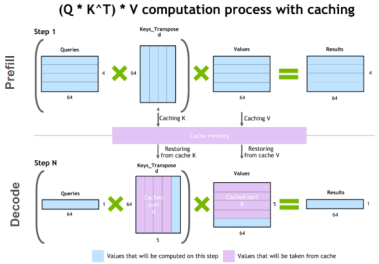
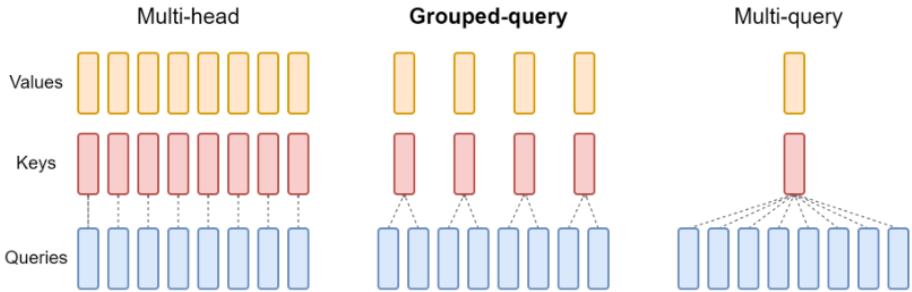


Figure 1: illustration of the key-value caching mechanism (from previous frame).

MHA, MQA, GQA: shrinking the KV cache



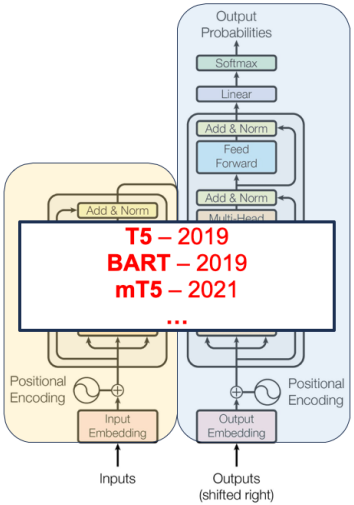
- **Multi-head attention (MHA)**: H separate query, key, and value heads.
- **Multi-query attention (MQA)**: single shared key and value heads for all query heads.
- **Grouped-query attention (GQA)**: shares key and value heads for each *group* of query heads.
- GQA conceptually interpolates between multi-head and multi-query attention.

Reference: Ainslie et al., "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," arXiv:2305.13245.

The LLM Era – Paradigm Shift in Machine Learning

BERT – 2018
DistilBERT – 2019
RoBERTa – 2019
ALBERT – 2019
ELECTRA – 2020
DeBERTa – 2020
…

Representation



GPT – 2018
GPT-2 – 2019
GPT-3 – 2020
GPT-Neo – 2021
GPT-3.5 (ChatGPT) – 2022
LLaMA – 2023
GPT-4 – 2023
…

Generation

Decoder-only LLMs (post-2017 trend)

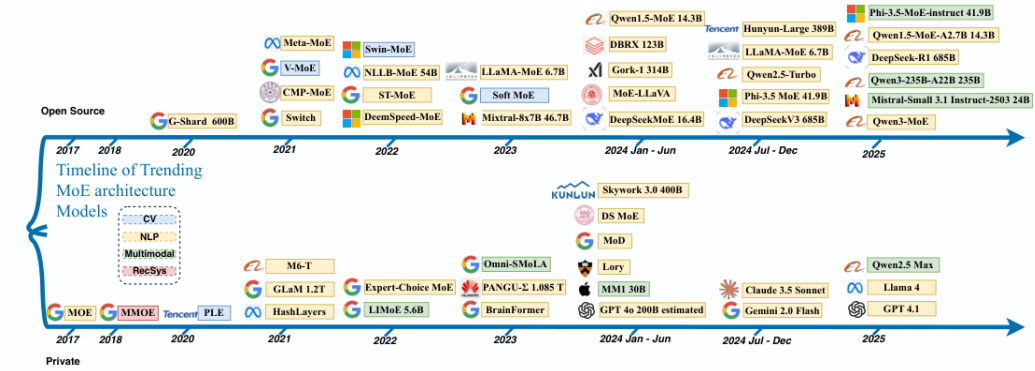
- GPT-style **causal (masked) self-attention**: each token attends only to the past.
- Single stack replaces separate encoder/decoder for many language (and vision–language) workloads.
- Encoder–decoder Transformers remain standard in **seq2seq** (MT, Whisper ASR, some VL captioning) where cross-attention helps.

Mixture of Experts (MoE)

- A dense feed-forward block always runs the same large MLP for every token. An MoE block replaces that MLP with several smaller expert MLPs.
- A lightweight router (gating network) looks at each token and picks one or a few experts to run. The rest stay idle for that token.
- Why it helps: you can increase total parameters (model capacity) without increasing active FLOPs per token as much as a fully dense model—similar spirit to "widen capacity, keep per-token work bounded."
- Routing can become unbalanced, communication-heavy in distributed training, or collapse onto a few experts unless training is regularized.

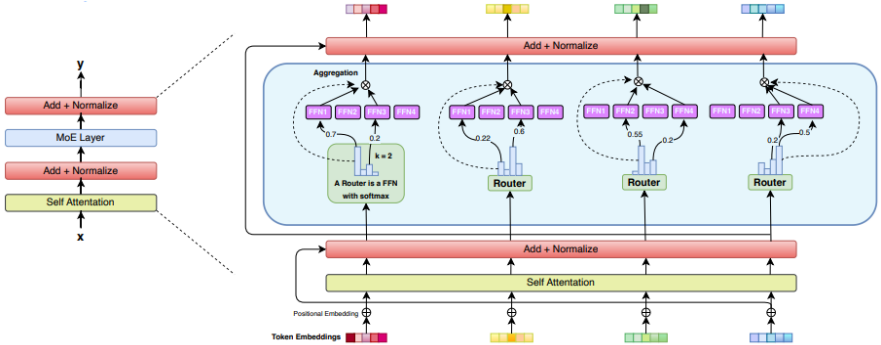
Sources: Zhang et al., "Mixture of Experts in Large Language Models," arXiv:2507.11181; Kranen & Nguyen, NVIDIA Technical Blog (Mar. 2024).

MoE milestones



Reference: Zhang et al., "Mixture of Experts in Large Language Models," arXiv:2507.11181.

MoE for a decoder-only Transformer

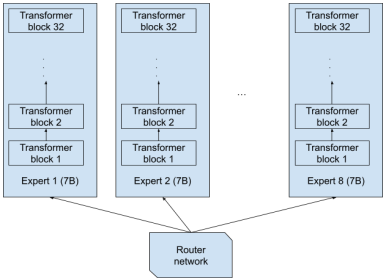


MoE for a decoder-only Transformer with $k = 2$ experts.

- Input **x** enters a **gating / router** that outputs scores for each expert.
- Only the top-k experts run their FFN; outputs are **combined** (often a weighted sum) for the residual stream.

Example: Mixtral 8×7B — a common misconception

- The name sounds like "eight separate 7B models." The **released architecture is not that**: experts are **not** eight full standalone stacks.
- Instead, think "**eight expert FFNs** per MoE layer, with **shared** attention and norms," and only a subset fires per token (here, **top-2** experts).



Reference: Kranen & Nguyen, "Applying Mixture of Experts in LLM Architectures," NVIDIA Technical Blog (2024), article.

MoE: Most Preferred Tokens By Experts

Reference: Kranen & Nguyen, "Applying Mixture of Experts in LLM Architectures," NVIDIA Technical Blog (2024), article.

Vision–Language Models: why pair vision with language?

- Strong **image embeddings** are useful, but many tasks still need a way to **say what you want** in words (“what species of bird is this?”, “read the sign in the corner”).
- **Language** makes it easy to specify new tasks without collecting a fresh classification label set for every concept—a path to flexible, user-steerable vision AI.
- **Nomenclature (informal):** an **LLM** is language-first; a **VLM** adds a **vision encoder** so the stack can **condition on images**; an **MLLM** usually means several modalities (image, video, audio) in one family.

Bandaru, *Vision Language Models* (blog, Aug. 2025), rohitbandaru.github.io/blog/Vision-Language-Models/.

Fusion Patterns: Where do image tokens meet the LLM?

- Early fusion: build one long token sequence (visual tokens + text tokens) and run attention over the mixture—what most "native" VLMs aim for today.
- Middle fusion: insert vision cross-attention inside some transformer blocks (common when adapting a legacy text-only stack such as early LLaMA 3.2).
- Late fusion: combine modalities only at the end—usually too weak for open-ended captioning/VQA because the LLM never gets rich visual state early enough.
- Images create many tokens; good designs compress visual length (pooling, grouping, tiling) before the LLM sees them.

Early Fusion approaches: MLP Adapter

- Freeze (or lightly train) a strong ViT; map its patch tokens through a small MLP projector into the LLM embedding width, then concatenate with text tokens.
- Simple, stable, and very common in open recipes (LLaVA, LLaMA 4-style early fusion, PaliGemma 2, DeepSeek-VL, . . .).

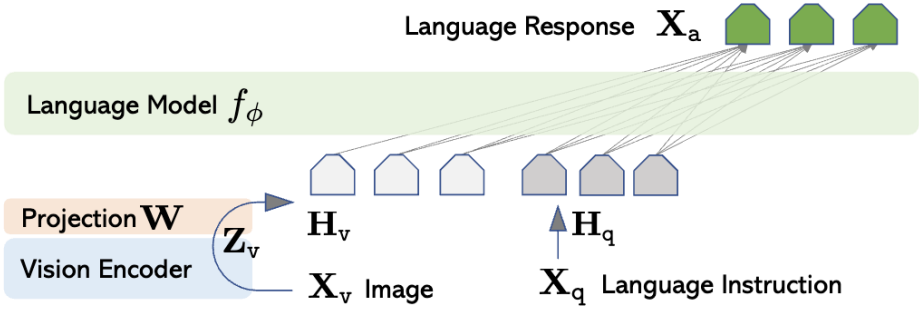


Diagram from Bandaru. Paper: Liu et al., LLaVA, arXiv:2304.08485.

Early Fusion approaches: Cross-Attention

- A small **adapter** uses learned query vectors that **cross-attend** to ViT tokens and output a **fixed small** set of visual summaries for the LLM.

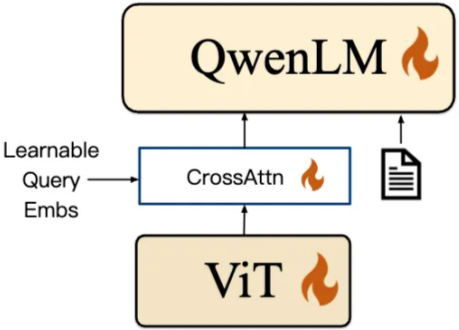


Diagram from Bandaru (Qwen-VL "position-aware" adapter).

Middle Fusion Approaches: Flamingo

- Keep a pretrained ViT and LLM frozen; insert Perceiver resamplers (compress variable tokens → fixed count) and gated cross-attention blocks inside the LLM stack.

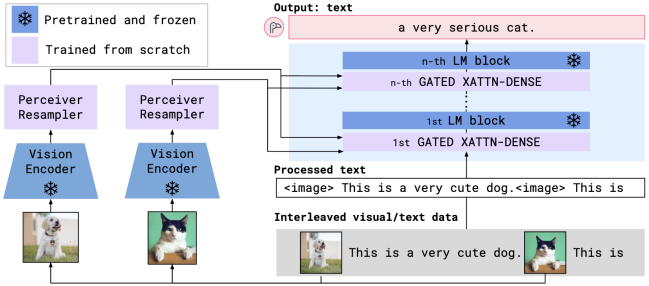


Diagram from Bandaru. Paper: Alayrac et al., Flamingo, arXiv:2204.14198.

Mid Fusion Approaches: LLaMA 3

- **ViT encoder:** 32-layer ViT trained CLIP-style for image-text alignment; features extracted from multiple layers.
- **Decoder:** Image cross attention in every fourth block (no gating on cross attention).
- **Mid fusion** suits LLaMA 3's text-first design; LLaMA 4 moves to early fusion (MLP adapter) for true multimodal training.

Summary: Transformers 2017 vs. 2026

Component	2017 Transformer	2026 consensus
Attention mechanism	Multi-Head Attention (MHA)	Grouped-Query Attention (GQA) / Multi-Head Latent Attention (MLA)
Position encoding	Absolute sinusoidal (or learned absolute)	Rotary Positional Embeddings (RoPE)
Normalization	Post-Layer Normalization (post-LN)	Pre-Layer Normalization with RMSNorm
Activation	ReLU in the FFN	SwiGLU / gated linear units
Bias terms	Biases in linear layers common	Bias-free or minimal-bias architectures
Parameter density	Dense (all parameters active per forward)	Sparse activation (e.g., Mixture-of-Experts)

Summary

- The slowest part is often getting data in and out of memory, not just the math.
- New models (2024–2026) are designed together, optimizing attention, normalization, position encoding, sparsity (MoE), and serving (memory, paging, kernels).
- The main goals: cut down on moving data, adjust compute to the input's length and type, and add more parameters without needing a lot more computation per token.

References and Further Reading

1. Vaswani et al., “Attention Is All You Need”, NeurIPS 2017, arXiv:1706.03762.
2. Su et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding”, arXiv:2104.09864.
3. Touvron et al., “LLaMA: Open and Efficient Foundation Language Models”, arXiv:2302.13971; “LLaMA 2: Open Foundation and Fine-Tuned Chat Models”, arXiv:2307.09288.
4. Shazeer, “Fast Transformer Decoding: One Write-Head is All You Need”, arXiv:1911.02150 (MQA).
5. Ainslie et al., “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints”, arXiv:2305.13245.
6. Fedus et al., “Switch Transformers: Scaling to Trillion Parameter Models”, JMLR 2022, arXiv:2101.03961.
7. Dao et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness”, NeurIPS 2022, arXiv:2205.14135; FlashAttention-2, arXiv:2307.08691.
8. Liu et al., “Visual Instruction Tuning” (LLaVA), arXiv:2304.08485.
9. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention”, arXiv:2309.06180 (vLLM).
10. Zhang et al., “Mixture of Experts in Large Language Models”, arXiv:2507.11181.
11. Kranen & Nguyen, “Applying Mixture of Experts in LLM Architectures,” NVIDIA Technical Blog, 2024, developer.nvidia.com/blog/....

References and Further Reading (cont.)

12. Bandaru, "Vision Language Models," blog, Aug. 2025, rohitbandaru.github.io/blog/Vision-Language-Models/.
13. Sorte (Medium), Transformer architectures in 2026: foundations and resources.
14. LLMOrbit (circular taxonomy of LLM systems).
15. Aman (Medium), Inside frontier open-weight LLM architectures.
16. MDPI survey, Mapping the LLM landscape (architectures and benchmarks).
17. MIT course notes, Transformers (6.390).
18. Shui, MHA vs. MQA vs. GQA.
19. DataHacker.rs, Modern transformer architectures (LLMs from scratch series).
20. Hugging Face blog, What changed in the Transformer architecture.
21. Tan, The crystallization of transformer architectures (2017–2025).
22. Bandaru, Transformer design guide (modern architecture).
23. NVIDIA, Mastering LLM techniques: inference optimization.
24. Gencc (Medium), KV cache, PagedAttention, and MLA.
25. PythonAlchemist, Attention variants explained (MHA, GQA, MQA, MLA, SWA, ...).

References and Further Reading (cont.)

26. Tri Dao, FlashAttention-3 (PDF); NeurIPS 2024 proceedings entry for FA3.
27. Lightly, Introduction to vision–language models.
28. Stanford CS224N, NLP with deep learning (course hub).
29. Marek Suppa, NLP 2026 (teaching resources).